Task 3 — Cross-Browser / Cross-Platform Matrix — Scenario D — Lê Phạm Kiều Duyên, 23127184

> **Status: EXECUTED 2026-08-02.** 26 of 28 cells captured — 20 Pass, 6 Fail. All **24 mandatory** cells are done and evidenced; the two outstanding cells (rows 27-28) belong to an optional old-WebKit extension and are not required by the coverage floor. Environments used: Windows 11 (Edge, Firefox) and an Android tablet as real local devices; macOS Safari, an Android phone and an iPhone 15 as real cloud devices via **Sauce Labs** — BrowserStack and LambdaTest were both tried first and found unusable on their free tiers (see the Tooling table). Verified with `matrix_coverage.py`, exit 0. See `.claude/skills/cross-platform-matrix/SKILL.md` for the method.

Scope: **D1-D4**, matching Task 1B's first four screens; D5/D6 are out of scope for Task 3. Screens: **D1** (Create Support Request, user) · **D2** (My Requests + detail, user) · **D3** (Admin Support Requests list) · **D4** (Admin request detail).

Why D1-D4 satisfies the brief. §6 opens with "Tasks 1B, 2, and 3 all operate on the **same three (or more) screens** of your chosen scenario," and Task 3 itself says "Test how your **three functions/screens** render and behave." Three is the floor, not the cap: D1-D4 are four of the six screens Task 1B executed, all drawn from the Scenario D screen list in §5 (D1-D4 verbatim), so the matrix exceeds the floor while staying inside the scenario's own screen set. Every coverage claim below is therefore made **per screen, four times over**, exactly as §6 requires.

Coverage floor required (§6 Task 3), **per screen** — the brief's own words:

> "Your matrix does not need every one of the 3×5×3 combinations, but it **must exercise every operating system at least once, every browser at least once, and every device class at least once, for each of the three screens.** State clearly which cells you covered and mark each **Pass / Fail**."

Required coverage values

- **OS (≥ 3):** Windows · macOS · Android **or** iOS
- **Browsers (≥ 5):** Chrome · Firefox · Safari · Edge · Opera (or Samsung Internet on mobile)
- **Device classes (3):** desktop · tablet · phone

Tooling (§6 Task 3)

The brief's order of preference, and the conditions attached to each:

Rung	Tool	Condition the brief attaches
1 (strongly preferred)	**BrowserStack** or **LambdaTest** trial	Named explicitly; obtaining trial access is the student's own responsibility (§6, §9)
2 (if the trial has expired)	Another cloud tool — **Sauce Labs** or **CrossBrowserTesting**	Same screenshot rule still applies in full

Rung	Tool	Condition the brief attaches
3 (if the trial has expired)	**Real physical devices**	Permitted only "provided each screenshot clearly shows the **browser / OS / device** name alongside the EMS URL" — on a physical device there is no cloud-lab banner, so an `about:` / device-info readout or a settings screen must be visible beside the app, or captured as a second image referenced from the same row

What was actually used, recorded as the run went rather than reconstructed afterwards:

Item	Value
Tool actually used	**BrowserStack Live free trial — attempted and found unusable for this matrix.** A real session was launched (macOS Sonoma + Safari 17.3, verified live) and the tier's own limit surfaced immediately: "*Each device is available for up to 1 minute during Free Trial*", confirmed by the in-session banner counting down to session end. The 30-minute allowance is not a usable block; it is spent in 1-minute slices. Every screen in this matrix (D1-D4) sits behind an EMS login, and typing an email and password on a remote machine consumes the whole minute before D1 is even reached — so the trial cannot capture a single cell, and holding several trial accounts does not help because the cap is **per session**, not per account. **LambdaTest was tried next and capped sessions at 2 minutes — the same problem.** What the matrix therefore ran on: rows 1-2 of every screen locally on the student's own Windows 11 PC (Edge, Firefox); rows 5 and 23 on a personally-owned Redmi Pad 2 tablet; and the macOS, Android-phone and iPhone rows on **Sauce Labs**, whose sessions are long enough to sign in and reach a screen — with the single exception of row 9 (D2 / macOS / Safari 18), captured on **TestingBot**. The Environment column records the kind per row
Account or trial identity	`lpkduyen23@clc.fitus.edu.vn`
Trial quota — **the binding constraint, and the run's main planning error**	BrowserStack's Live trial showed **30 minutes total** (`Account & Profile → Overview → Plan Details`, 2026-08-02), one-time and non-renewable. The mistake was planning against that headline number: the binding limit is not the total but the **per-session cap** (1 minute on BrowserStack, 2 on LambdaTest), and it was not checked until a session was already running. Both caps should have been read *before* the first launch, because they invalidate a plan built around 30-minute blocks. Two consequences that survived into the actual run: **(a)** no metered minutes were spent rehearsing — the dry run happened on the local Windows machine; **(b)** every cell that could be captured on hardware the student already owns was, keeping paid-tier time for environments that cannot be borrowed. BrowserStack's separate **Automate** trial (100 minutes) is not a substitute: it captures the viewport only, with no address bar and no device banner, so its screenshots fail evidence requirements 1 and 2 by construction
Mobile OS decided (Android or iOS)	**Both.** Android for rows 4-5 of each screen (phone + tablet), iOS for row 6 (phone, real device). The brief requires three OS; this matrix carries four — Windows, macOS, Android, iOS — so `--os` in the verification command must list all four
EMS accounts actually used	**User side (D1-D2): the student's own account**, displayed in EMS as `DUYÊN LÊ PHẠM KIỀU / 23127184@student.hcmus.edu.vn / 23127184`. **Admin side (D3-D4): the `TLA` admin account issued by the lecturer** — *not* the `admin@gmail.com` / `Admin@123` pair printed in the brief §"Admin account", which was superseded by the account actually handed out. Recorded here because a reader comparing the screenshots against the brief would otherwise flag the mismatch as an error. The same two accounts are used in every environment, so any difference between cells is attributable to the browser and not to the data

Item	Value
Overlay burned into every capture	**`23127184 · lpkduyen23@clc.fitus.edu.vn · <OS> · <browser+version> · <device class>`** — e.g. `23127184 · lpkduyen23@clc.fitus.edu.vn · Windows 11 · Edge 151.0.4129.59 · desktop`. The student ID leads, then the student's FITUS email, then the environment. **A deliberate choice, recorded because it does not match the brief's wording literally:** §6 asks for the username "in the form `MSSV@....edu.vn`", and this student also holds `23127184@student.hcmus.edu.vn`, which matches that pattern exactly and which EMS itself displays on D4. The captures were re-stamped to that address at one point and then reverted, on the student's decision, to the FITUS address with the ID carried alongside — both identify her unambiguously and the ID is present either way. Flagged here so a grader comparing the overlay against §6's exact phrasing sees the reasoning rather than an oversight. The environment tail satisfies the separate requirement that the browser/OS/device **name** be visible in the pixels — on a personal machine there is no cloud-lab banner to supply it, and `edge://version` cannot be opened from the command line to serve as a companion image. The same address is used on the §7 Google Form submissions

Engine reminder (do not fill the matrix with one engine wearing five brand names)

Engine	Browsers in this matrix
Blink	Chrome, Edge, Opera, Samsung Internet
Gecko	Firefox
WebKit	Safari (and, by default, every iOS browser regardless of brand, unless the row states an EU/UK BrowserEngineKit exception)

Force at least one Gecko and one WebKit cell. Note the engine per row.

Screenshot requirements (§6 Task 3 + §12, mandatory)

One screenshot per cell — all 24 mandatory cells, Pass rows included. §6: "Capture a screenshot for **every cell** in your matrix; each screenshot must overlay your username in the form **MSSV@....edu.vn** (your student-ID email)." A Fail additionally needs "a short note on the defect (overflow, overlap, broken layout, unreadable text, non-responsive control, etc.)" — recorded in the row's Note cell and in "Failures" below. §12 lists the cross-platform screenshots among the artefacts TAs verify as non-fabricable, and requires the email overlay "alongside the EMS URL and the browser/OS/device identity" — that combination is required of **every** capture, not only of the physical-device fallback where §6 words it.

Every screenshot must show, in the image itself:

1. the EMS URL,
2. the browser / OS / device identity (the lab's own banner, or an about page beside the app),
3. **your student-ID email overlay in the form `MSSV@....edu.vn`** — burn this into the image (annotation tool, lab's own overlay feature, or a visible watermark) — this is a hard anti-cheat requirement, not optional,
4. the screen in the state being claimed.

An overlay that is present but illegibly small, low-contrast, or cropped does not satisfy the rule; a caption in this markdown file or in the filename does not satisfy it either — it must be in the pixels. **A capture missing the overlay is not evidence:** its cell reverts to `Not executed` rather than being scored on the strength of the rest of the image.

Name files: `<Screen>_<OS>_<Browser>_<Device>.png`, saved under `reports/evidence_task3/`. If the capture tool emits `.jpg` instead, keep the same stem and change the extension in the **file and the**

Evidence cell together — the verifier resolves the Evidence cell against the real filename.

Environment kind — record which, per cell

The brief (§6 Task 3) requires the emulator / simulator / real-device distinction to be understood and applied; the **Environment** column in the matrix is where each cell records which one actually ran. Use exactly one of these four strings, and never leave the column blank:

`real device (local)` · `real device (cloud)` · `emulator` · `simulator`

Kind	What it proves	What it does not prove
Responsive mode (DevTools)	Layout at a resized viewport, one engine	Nothing engine- or OS-specific
Emulator (Android)	Real OS image, virtualised GPU	Real font rendering, real performance
Simulator (iOS)	Mac-native reimplementation	Real WebKit build quirks
Real device (own or cloud lab)	The actual thing	—

DevTools responsive mode is not an acceptable environment for any row in this matrix. Resizing a Windows Chrome window to phone dimensions is still Windows, still one engine, and still one device class — it cannot satisfy the phone or tablet rows, and a screenshot of it would misreport what was tested. If no real Android device or cloud real-device slot can be reached, the honest record is `emulator` in the Environment column, not a resized desktop browser labelled "phone".

Matrix

> **Acceptance rule applied to every captured cell — all five had to hold, or the cell was recaptured: (1)** the EMS URL is visible in the image · **(2)** the browser / OS / device identity is visible in the image (lab banner, or device-info readout beside the app on a physical device) · **(3)** the student-ID overlay is burned into the pixels, legibly, without covering the content being judged · **(4)** the screen is in the state the row claims (resting state for a `Pass`, the defective state for a `Fail`) · **(5)** the file is saved under `reports/evidence_task3/` under the filename in the row's Evidence cell. A cell failing any of the five stayed `Not executed` rather than being scored on the strength of the rest of the image — which is why row 22 was recaptured after its first image turned out to show an inherited sidebar state rather than the default one.

#	Screen	OS	Browser	Engine	Device class	Environment	Result	Evidence	Note
1	D1	Windows 11	Edge	Blink	desktop	real device (local)	Pass	`D1_Windows_Edge_desktop.png`	Form renders complete at 1382×736; no overflow, overlap or clipping
2	D1	Windows 11	Firefox	Gecko	desktop	real device (local)	Pass	`D1_Windows_Firefox_desktop.png`	Gecko renders the form identically to Blink; no engine-specific difference
3	D1	macOS	Safari	WebKit	desktop	real device (cloud)	Pass	`D1_macOS_Safari_desktop.png`	Sauce Labs Live, macOS Sequoia + Safari 18. Form renders complete. Captured under a second student-side account (`NM`) rather than `DLPK`; D1 is an empty create-form whose content does not vary by account, so the comparison holds
4	D1	Android	Chrome	Blink	phone	real device (cloud)	Pass	`D1_Android_Chrome_phone.png`	Sauce Labs Mobile Real, Samsung Galaxy S23 FE / Android 16. Form reflows to one column; no overflow or clipping
5	D1	Android	Opera	Blink	tablet	real device (local)	Pass	`D1_Android_Opera_tablet.png`	Redmi Pad 2 / Android 16, Opera. Form renders at full width with no reflow problems
6	D1	iOS 17+	Safari	WebKit	phone	real device (cloud)	Pass	`D1_iOS_Safari_phone.png`	Sauce Labs Mobile Real, iPhone 15 / iOS 26.5. Mobile WebKit reflows the form to one column correctly
7	D2	Windows 11	Edge	Blink	desktop	real device (local)	Pass	`D2_Windows_Edge_desktop.png`	List, search, status filter and pagination all render; 2 resolved requests shown
8	D2	Windows 11	Firefox	Gecko	desktop	real device (local)	Pass	`D2_Windows_Firefox_desktop.png`	List, filter and pagination match the Edge capture cell for cell
9	D2	macOS	Safari	WebKit	desktop	real device (cloud)	Pass	`D2_macOS_Safari_desktop.png`	TestingBot Live, macOS Sonoma + Safari 18. WebKit renders the list identically to Blink and Gecko
10	D2	Android	Chrome	Blink	phone	real device (cloud)	Pass	`D2_Android_Chrome_phone.png`	Re-captured under `DLPK` after a first attempt on a second account showed the empty state — the populated list is what the desktop cells show, so this keeps the comparison like-for-like
11	D2	Android	Opera	Blink	tablet	real device (local)	Pass	`D2_Android_Opera_tablet.png`	List, search, status filter and pagination all render; matches the desktop captures cell for cell
12	D2	iOS 17+	Safari	WebKit	phone	real device (cloud)	Pass	`D2_iOS_Safari_phone.png`	Reached via the page's own Back control after Safari's address-bar autocomplete kept resolving typed URLs to the previously visited page. List renders both requests
13	D3	Windows 11	Edge	Blink	desktop	real device (local)	Pass	`D3_Windows_Edge_desktop.png`	Admin list renders: Pending/Resolved counters, filter panel, results table
14	D3	Windows 11	Firefox	Gecko	desktop	real device (local)	Pass	`D3_Windows_Firefox_desktop.png`	Counters, filters and table all render; date inputs differ in format only — see Platform differences
15	D3	macOS	Safari	WebKit	desktop	real device (cloud)	Pass	`D3_macOS_Safari_desktop.png`	Sauce Labs Live, macOS Sequoia + Safari 18, `TLA` admin. Sidebar, counters, filter panel and results table all render
16	D3	Android	Chrome	Blink	phone	real device (cloud)	**Fail**	`D3_Android_Chrome_phone.png`	`broken-layout` — at 1080 px phone width the admin sidebar keeps its desktop width and does not auto-collapse, squeezing the page into roughly a quarter of the viewport: the heading wraps one word per line and the results table is pushed off-screen. Collapsing the sidebar by hand restores a usable layout, which confirms the defect is the **default state**, not the content
17	D3	Android	Opera	Blink	tablet	real device (local)	Pass	`D3_Android_Opera_tablet.png`	**The cell that locates the responsive breakpoint.** Same screen, same OS family and same Blink engine as the failing phone cells — but at tablet width the sidebar sits at its normal size and the whole page is readable. So the admin-layout defect is bounded by viewport width, not caused by Android or by Blink
18	D3	iOS 17+	Safari	WebKit	phone	real device (cloud)	**Fail**	`D3_iOS_Safari_phone.png`	`broken-layout` — reproduces the Android defect exactly on a different engine: expanded sidebar, heading wrapping one word per line, table off-screen. **This is the cell that proves the defect is EMS's responsive CSS and not a Blink quirk.**
19	D4	Windows 11	Edge	Blink	desktop	real device (local)	Pass	`D4_Windows_Edge_desktop.png`	Admin detail on request #30 — the student's own request, so no third party's data enters the evidence set
20	D4	Windows 11	Firefox	Gecko	desktop	real device (local)	Pass	`D4_Windows_Firefox_desktop.png`	Admin detail on request #30; matches the Edge capture

#	Screen	OS	Browser	Engine	Device class	Environment	Result	Evidence	Note
21	D4	macOS	Safari	WebKit	desktop	real device (cloud)	Pass	`D4_macOS_Safari_desktop.png`	Reached by clicking a list row rather than retyping the URL. Shows request #43, not the #30 used on Windows — the admin detail layout is what this row compares, and it is identical
22	D4	Android	Chrome	Blink	phone	real device (cloud)	**Fail**	`D4_Android_Chrome_phone.png`	`broken-layout` — same defect as D3, and worse here. A first capture was taken with the sidebar still collapsed from D3 and looked fine; **re-loading the URL fresh** reproduced the real default state: the expanded sidebar leaves a strip so narrow that the request title wraps **one word per line**. The re-load is what turned this cell from an unproven Pass into a confirmed Fail
23	D4	Android	Opera	Blink	tablet	real device (local)	Pass	`D4_Android_Opera_tablet.png`	Request #47, loaded fresh by URL so the sidebar state is genuinely the default. Detail cards render correctly — the same screen that wraps one word per line on both phones
24	D4	iOS 17+	Safari	WebKit	phone	real device (cloud)	**Fail**	`D4_iOS_Safari_phone.png`	`broken-layout` — loaded fresh by URL, so this is the true default state. Request title wraps one word per line, identical to the Android/Chrome result. Four Fails across two engines and two OSes, all four on the two admin screens and none on the two user screens
25	D1	macOS Monterey	Safari 15	WebKit	desktop	real device (cloud)	**Fail**	`D1_macOS_Safari15_desktop.png`	`broken-layout` — **the entire stylesheet fails to apply.** Default serif type, unstyled native form controls, blue underlined nav links, logo at intrinsic size overflowing the viewport. The same screen on Safari 18 (row 3) is pixel-perfect, so this is version-specific, not WebKit-wide
26	D2	macOS Monterey	Safari 15	WebKit	desktop	real device (cloud)	**Fail**	`D2_macOS_Safari15_desktop.png`	`broken-layout` — same total stylesheet failure as row 25, so the fault is app-wide rather than confined to one screen. Status chips render as bare text lines, the search box as an unstyled native input, "Create request" as a text link
27	D3	macOS Monterey	Safari 15	WebKit	desktop	Not executed	Not executed		capture as `D3_macOS_Safari15_desktop.png`
28	D4	macOS Monterey	Safari 15	WebKit	desktop	Not executed	Not executed		capture as `D4_macOS_Safari15_desktop.png`

Rows 25-28 — the old-WebKit extension, added after the 24-cell matrix was complete. These are *not* padding and they are not needed for the coverage floor, which rows 1-24 already meet on every screen. They exist because this file's own rule for going beyond the floor is to spend extra rows on **the combinations most likely to break**, and an older WebKit build is exactly that. The bet paid off immediately: Safari 15 fails where Safari 18 passes, which is a defect the 24-cell matrix could not have found because it pins Safari to its latest version. Note what this does *not* claim — one old browser version failing is not evidence about WebKit in general, and row 3 (Safari 18, Pass) is the control that makes the distinction visible.

Row plan (covering array, not full cross-product). The brief states the coverage floor **per screen**, and no cell can carry more than one browser, so a screen cannot cover 5 required browsers in fewer than 5 rows: the floor is `max(3 OS, 5 browsers, 3 device classes) = 5` cells per screen, hence a floor of **20 cells across D1-D4**. **This matrix runs 24, one above the floor per screen.** The sixth row is **iOS + Safari + phone on a real device**, added deliberately: at the bare floor, three of the five required brands (Chrome, Edge, Opera) are Blink, so two of the three engines would be exercised exactly once per screen and mobile WebKit — the combination most likely to break this layout — not at all. The extra row buys a fourth OS, a second WebKit environment, and the real-device-versus-simulator distinction the brief asks to be understood. The same 6-tuple is applied to each screen so the coverage argument is identical on every one of them, and so that one cloud-lab session can capture all four screens before it is closed. Checked against the floor: OS {Windows, macOS, Android} = 3/3 · browsers {Edge, Firefox, Safari, Chrome, Opera} = 5/5 · device classes {desktop, phone, tablet} = 3/3 · engines {Blink, Gecko, WebKit} = 3/3, so the matrix is not one engine wearing five brand names.

Three of the five brands are Blink (Chrome, Edge, Opera) — only Firefox (Gecko) and Safari (WebKit) add real engine diversity. That is why the rows added beyond the floor went to WebKit rather than to a fourth Blink brand: row 6 of each screen (mobile WebKit) and then rows 25-28 (an older desktop WebKit build). Both bets found defects a Blink row would not have.

Each row's capture filename was assigned before the capture was taken, so no screenshot ever had to be reconciled against a row afterwards. `stamp_evidence.py` derives the filename from this table's own cells, which is what keeps image and row from drifting apart. On iOS, note the WebKit-by-default rule: every browser is WebKit regardless of brand unless the row documents an EU/UK BrowserEngineKit exception.

The run order that collapsed the metered cloud-lab work into as few launches as possible is recorded in `docs/cross_platform/00_Run_Plan.md` §6; the local Windows rows cost nothing and ran first.

Coverage achieved

Screen	OS covered	Browsers covered	Device classes covered	Engines covered	Missing
D1	Windows, macOS, Android, iOS (4/4)	Edge, Firefox, Safari, Chrome, Opera (5/5)	desktop, phone, tablet (3/3)	Blink, Gecko, WebKit (3/3)	—
D2	Windows, macOS, Android, iOS (4/4)	Edge, Firefox, Safari, Chrome, Opera (5/5)	desktop, phone, tablet (3/3)	Blink, Gecko, WebKit (3/3)	—
D3	Windows, macOS, Android, iOS (4/4)	Edge, Firefox, Safari, Chrome, Opera (5/5)	desktop, phone, tablet (3/3)	Blink, Gecko, WebKit (3/3)	—
D4	Windows, macOS, Android, iOS (4/4)	Edge, Firefox, Safari, Chrome, Opera (5/5)	desktop, phone, tablet (3/3)	Blink, Gecko, WebKit (3/3)	—

Result: 26 of 28 cells executed — 20 Pass, 6 Fail. The two outstanding cells are rows 27–28 (D3/D4 on Safari 15), which belong to the optional old-WebKit extension and are not required by the coverage floor; every one of the 24 mandatory cells is captured and evidenced.

What the six failures establish, and what they do not. Four of them (D3 and D4 on both phones) share one cause, and the matrix isolates it by elimination rather than by assertion: not the engine, because the defect appears on Blink *and* WebKit; not Android, because the tablet runs the same Android 16 and the same Blink and passes; not small screens generally, because D1 and D2 pass on those same two handsets. What remains is a missing responsive breakpoint in the admin area, with the threshold sitting between the iPhone 15's 1179 px and the Redmi Pad 2's tablet width. The remaining two failures are a separate, version-specific fault: Safari 15 drops the stylesheet entirely on both D1 and D2, while Safari 18 renders both perfectly — so this is about one old browser build, not about WebKit.

Verify with (run from `HW03/`):

```
python .claude/skills/cross-platform-matrix/scripts/matrix_coverage.py
docs/04_Task3_Cross_Platform_Matrix.md \
  --os "Windows,macOS,Android,iOS" \
  --browsers "Chrome,Firefox,Safari,Edge,Opera" \
  --devices "desktop,tablet,phone" \
  --evidence-root reports/evidence_task3/
```

`--os` must name **exactly the operating systems the rows actually use** — every value listed is treated as required, and every value *omitted* goes unchecked. All four are listed above because the sixth row of each screen is iOS: dropping `iOS` from the list still exits 0, but it silently leaves the four iOS rows unverified, which is the more dangerous failure of the two. If the mobile-OS decision changes (see `docs/cross_platform/00_Run_Plan.md` §4 item 4), change the rows **and** this command together. A clean run prints `4/4`, `5/5`, `3/3` per screen with no `MISSING`, at least two engines per screen, and ends `OK -- coverage floor met on every screen, evidence resolves` with exit code 0. It was verified twice: once while the matrix was still empty, and again after execution — both with and without `iOS` in the list — so any later failure is a real regression, not scaffolding noise.

Failures (grouped by classification)

§6 Task 3: "Attach screenshots for any rendering/layout **Fail** with a short note on the defect (overflow, overlap, broken layout, unreadable text, non-responsive control, etc.)." One row here per `Fail` cell in the matrix above — the same screenshot, plus the note.

Use the fixed vocabulary: `overflow` · `overlap` · `clipping` · `unreadable` · `unresponsive-control` · `missing-asset` · `feature-unsupported` · `broken-layout`. Reproduce any Fail once (ideally a fresh session) before logging it, and cross-check the image itself — cloud labs can serve a stale screenshot mid-load. Separate a genuine incompatibility from a legitimate platform difference (native date-input locale format, platform scrollbars, system font substitution, the mobile URL bar changing viewport height) — the latter goes in the next section, not here.

Cell #	Screen	OS / Browser / Device	Classification	Short note on the defect	Evidence	Findings-log ID
#16	D3	Android 16 / Chrome / phone	`broken-layout`	Admin sidebar keeps its desktop width at 1080 px and never auto-collapses; content is squeezed into ~¼ of the viewport, heading wraps one word per line, results table pushed off-screen. Collapsing the sidebar by hand restores a usable layout, proving the fault is the default state	`D3_Android_Chrome_phone.png`	D-020
#22	D4	Android 16 / Chrome / phone	`broken-layout`	Same defect, worse: request title wraps one word per line. Only visible on a **fresh URL load** — the first capture inherited D3's collapsed sidebar and looked healthy	`D4_Android_Chrome_phone.png`	D-020

Cell #	Screen	OS / Browser / Device	Classification	Short note on the defect	Evidence	Findings-log ID
#18	D3	iOS 26.5 / Safari / phone	`broken-layout`	Reproduces cell 16 exactly on WebKit. **This pairing is what rules out a Blink-specific cause**	`D3_iOS_Safari_phone.png`	D-020
#24	D4	iOS 26.5 / Safari / phone	`broken-layout`	Reproduces cell 22 on WebKit, loaded fresh by URL	`D4_iOS_Safari_phone.png`	D-020
#25	D1	macOS Monterey 12 / Safari 15 / desktop	`broken-layout`	**The whole stylesheet fails to apply.** Serif default type, native unstyled controls, underlined blue links, logo at intrinsic size overflowing the viewport. Reproduced in a second independent session	`D1_macOS_Safari15_desktop.png`	D-021
#26	D2	macOS Monterey 12 / Safari 15 / desktop	`broken-layout`	The same total stylesheet failure on a second screen, which is what establishes it as app-wide rather than one broken page: status chips render as bare text lines, the search box as an unstyled native input, "Create request" as a plain text link	`D2_macOS_Safari15_desktop.png`	D-021

Two further defects found while probing older WebKit, recorded here but *not* given matrix cells because they occur on the sign-in screen, which is outside this matrix's D1-D4 scope:

Where	Classification	What was seen	Evidence
Sign-in, macOS Monterey / **Safari 15**	`broken-layout`	Same total stylesheet failure as cell 25 — this is where the defect was first noticed, before it was confirmed on D1	`reports/evidence_task3/safari15_defect/safari15_login_unstyled_repro.jpg`
Sign-in, macOS Monterey / **Safari 16**	`missing-asset` / blocking	CSS *does* apply, but the **Email and Password inputs never render** — only the Login button and two links appear. **A user on Safari 16 therefore cannot sign in at all**, which makes every other screen unreachable. Verified across three captures spanning ~2 minutes with no change, and a scrollbar is present, so the DOM has laid out content that is not being painted — this is a render fault, not an unfinished load. **Reproduced in a second independent session (2026-08-02), and again after a full page reload inside it**	`reports/evidence_task3/safari15_defect/safari16_login_fields_missing.jpg`, `safari16_login_fields_missing_session2_reload.jpg`

On severity, stated carefully: the Safari 16 sign-in fault is the most serious thing found in Task 3 — it blocks access outright rather than degrading appearance. It was first observed in a single cloud session, and was recorded with that limit visible rather than asserted more strongly than the evidence allowed. **A second, independent Sauce Labs session on 2026-08-02 reproduced it**, and a full page reload within that session reproduced it a third time, so it now meets the same reproduction bar the Safari 15 fault already met. The residual limit is breadth, not existence: both sessions used the same Sauce Labs Safari 16 image, so this is a confirmed fault on that build rather than a measurement across every Safari 16 in the wild.

Handoff to §7 (Bug & Usability Findings)

Every genuine incompatibility found here must be reported **twice**, per §7 of the brief:

1. **Google Form** — https://forms.gle/CJQFQCAXcsDbXDMM9, submitted from the same student-ID email that is burned into the screenshot overlays, so the submission is attributable.
2. **`docs/05_Bug_Usability_Findings_Log.md`** — typed `Bug`, with the matrix cell (screen, OS, browser, device class, environment kind) as its reproduction context and the cell's screenshot as its evidence reference. Record the form-submission timestamp in the log's own column; §7 warns the aggregated file and the form submissions must be consistent and the TA may cross-check counts.

New IDs start at **D-020** (D-001…D-019 are taken by Task 1B). **D-013, D-014 and D-018 are retired and must never be reused.** Task 2 also draws from D-020 onward — allocate an ID only at the moment a finding is actually written, never reserve a block in advance, and record the ID back into the "Findings-log ID" column above.

Platform differences observed (not defects)

Record legitimate platform behaviour here so a reviewer sees it was considered, not missed — e.g. a native `From date` / `To date` input on the Support Requests filter panel (D3) rendering in the browser's own locale format is expected, not a bug (see `docs/01_Task1A_Shared_GUI_Checklist.md` IA02-11).

#	Where	Observed	Why this is not a Fail
1	D3, `From date` / `To date` on the filter panel	Edge renders the empty native date input as `mm/dd/yyyy`; Firefox renders the same input as `dd / mm / yyyy`, with spaces around the separators	`<input type="date">` is drawn by the browser, not by the application, and each browser formats the placeholder to its own locale convention. EMS sends identical markup to both. Anticipated in advance by this file's own example and by checklist item IA02-11 — recorded here rather than logged as a defect
2	All screens, address bar	Edge shows the full `https://prod-dev.ems-fitus.cloud/…`; Firefox trims the `https://` scheme and shows `prod-dev.ems-fitus.cloud/…`	Browser-chrome behaviour, entirely outside the application. Noted because it affects the *evidence*, not the product: the host and path stay legible in every Firefox capture, so requirement 1 (EMS URL visible in the pixels) is still met